# ScrewIT

Technical Prototype Documentation

How the system works, the technology behind it, and how it solves the problem statement

| | |
|---|---|
| Problem Statement | SIH 2026 · Problem 26099 — AI-Driven Standardization & Harmonization of Material Codes across CPSEs |
| Ministry | Ministry of Petroleum & Natural Gas |
| Theme | Clean & Green Technology / Smart Automation |
| Prototype | ONOMC — One Nation, One Material Code |

# 1. Problem Statement & Solution Overview

## 1.1 The problem

Central Public Sector Enterprises (CPSEs) across Oil & Gas, Power, and Steel each run independent SAP/ERP instances. The same physical item — e.g. a **6 inch, Class 300, RF flanged gate valve** — is created as different material codes, different units of measure, and inconsistently worded free-text descriptions in every company's catalog. Consequences:

- **15–30% catalog redundancy** — duplicate material masters inflate inventory and procurement records in every CPSE.
- **Lost bulk-procurement leverage** — with no visibility into aggregate demand across companies, CPSEs buy the same item separately at higher unit prices.
- **Error-prone reconciliation** — every cross-CPSE sourcing attempt today is a manual, spreadsheet-driven matching exercise.

## 1.2 The solution in one line

ScrewIT is an **AI material-code harmonization platform** that ingests each CPSE's raw material catalog, detects duplicate items across companies with a similarity-scoring engine, routes them through a human review workflow, and publishes a single **national material code (ONOMC)** that maps bidirectionally back to every source system — *without replacing or disrupting any legacy ERP*.

## 1.3 How the prototype solves the problem (end-to-end flow)

| Stage | What the prototype does | Where it lives (code) |
|---|---|---|
| 1 · Ingest | CSV/catalog upload is parsed client-side, validated against required columns, previewed, then bulk-inserted into the materials table. | src/pages/DataIngestionPage.tsx (PapaParse) → convex/ingestion.ts (bulkInsert) |
| 2 · Match | The AI matching engine normalizes descriptions, scores every cross-CPSE pair, clusters items above threshold, and drafts a national code per cluster. | convex/matching.ts — generateMatchCandidates (action) |
| 3 · Review | Pending mappings appear in a review queue with side-by-side comparison and confidence scores. A reviewer approves, edits-and-approves, or rejects each one. | src/pages/ReviewQueuePage.tsx → convex/review.ts |
| 4 · Harmonize | Approved clusters become published national codes; analytics computes duplicate clusters, CPSE participation, and estimated procurement savings. | convex/analytics.ts, src/pages/AnalyticsPage.tsx |
| 5 · Sync back | Approved mappings are pushed back to each simulated CPSE ERP connector (RFC/BAPI / REST / OData) with progress tracking per company. | convex/integration.ts, src/pages/SystemIntegrationPage.tsx |
| 6 · Audit | Every create/approve/reject/edit is written to an append-only audit log with full before/after state — viewable with filters and JSON diff. | convex/analytics.ts (recentActivity), src/pages/AuditTrailPage.tsx |

# 2. Technology Stack (as built in this prototype)

| Layer | Technology | Role in the prototype |
|---|---|---|
| Frontend | React 19 + TypeScript (SPA) | 7-page governance dashboard: Overview, Materials, Review Queue, Analytics, Audit Trail, Data Ingestion, System Integration. |
| Build tool | Vite 8 | Dev server with HMR; production bundling (tsc -b && vite build). |
| Routing | React Router DOM v7 | Client-side navigation between the dashboard pages. |
| Backend / DB | Convex (TypeScript serverless backend) | Managed database + queries + mutations + actions. Real-time data sync to the UI without hand-written REST endpoints. |
| Matching engine | TypeScript action + string-similarity (Dice coefficient) | Pairwise text similarity, classification bonuses, technical-attribute conflict penalties, union-find clustering. |
| CSV ingestion | PapaParse | Browser-side CSV parsing, header validation, row-level error reporting. |
| Styling | Hand-written CSS (custom design system) | Enterprise navy/amber theme over a fixed refinery-dusk photographic backdrop. |
| Code quality | TypeScript strict mode, Oxlint | Type-safe end-to-end (shared types from Convex schema) and lint-clean. |

**Note for evaluators:** the prototype deliberately consolidates the production architecture (API gateway, FastAPI ML service, PostgreSQL + pgvector, Kafka, Keycloak — see the technical proposal) into Convex's single serverless backend. This keeps the demo self-contained and instantly runnable (*npm run dev:all*) while preserving every functional capability: ingestion, matching, review, analytics, sync-back, and auditability.

## 2.1 Data model (four core tables)

```
cpseMaterials        one row per CPSE material, as-is from their ERP
  cpseId, cpseName, sourceMaterialCode, sourceDescription,
  uom, classificationCode, technicalAttrs(JSON), ingestedAt

nationalMaterials    the harmonized national master (ONOMC codes)
  nationalCode, standardDescription, standardUom,
  standardClassification, status(draft|approved), createdAt

materialMappings     bidirectional link: national code <-> CPSE source
  nationalCode, cpseId, sourceMaterialCode,
  matchType(exact|near_duplicate|functional_equivalent),
  confidenceScore(0-100), reviewStatus(pending|approved|rejected),
  reviewedBy, reviewedAt, syncedAt

auditLog             append-only, immutable trail
  entityType, entityId, action, actor,
  beforeState(JSON), afterState(JSON), timestamp
```

# 3. The AI Matching Engine — Technical Walkthrough

The engine lives in **convex/matching.ts** and runs as a Convex **action** (generateMatchCandidates). It is a deterministic, explainable pipeline — every score it produces can be traced back to its inputs, which matters for auditability in a government context.

## 3.1 Pipeline

```
Load materials + existing mappings (queries)
        |
        v
Filter out already-mapped materials
        |
        v
Pairwise scoring of every cross-CPSE pair        threshold >= 0.55
  score = Dice(normalizedDescA, normalizedDescB)   0 .. 1
        + 0.08  if classificationCode matches        (category bonus)
        - 0.15  if technical attrs conflict          (size/pressure/etc.)
        |
        v
Union-Find clustering of scored pairs            transitive merge
        |
        v
For each cluster with >= 2 members:
  - canonical description  = longest member description
  - standard UoM           = most frequent member UoM
  - standard classification= most frequent member class
  - nationalCode           = 'ONOMC-' + FNV-1a hash of sorted member keys
  - matchType              = exact(>0.85) | near_duplicate(>=0.70) | functional_equivalent
  - confidenceScore        = score * 100 (1 decimal)
        |
        v
createCluster mutation:
  insert nationalMaterials (status = draft) + one materialMappings row
  per member (reviewStatus = pending) + auditLog entry
```

## 3.2 Worked example (from the seeded demo data)

| CPSE | Source code | Description as entered | Result |
|---|---|---|---|
| CPCL | MAT-00123 | 6 inch, Class 300, RF flanged gate valve | Clustered together → national code ONOMC-XXXXX drafted, 3 pending mappings with high confidence. |
| PowerGen Ltd | PG-VLV-889 | Gate valve, flanged, 6", class 300#, rising stem | Same cluster (near_duplicate). |
| SteelCo India | SC/VALVE/045 | 6 inch gate valve CL300 RF A216 WCB | Same cluster (near_duplicate). |

The three descriptions differ in wording, quoting, and abbreviation — a keyword match would miss them. The Dice coefficient on *normalized* text (lowercased, punctuation stripped, whitespace collapsed) scores these pairs high enough to cluster, while the technical-attribute conflict penalty prevents a 6-inch CL300 valve from ever being merged with a 4-inch CL150 one even if the wording is similar.

## 3.3 Why this design

- **Explainable:** each mapping carries its numeric confidence and match type — reviewers see *why* items were grouped.

• **Conservative by default:** attribute conflicts (size, pressure class, schedule, grade) actively veto bad merges instead of relying on text alone.

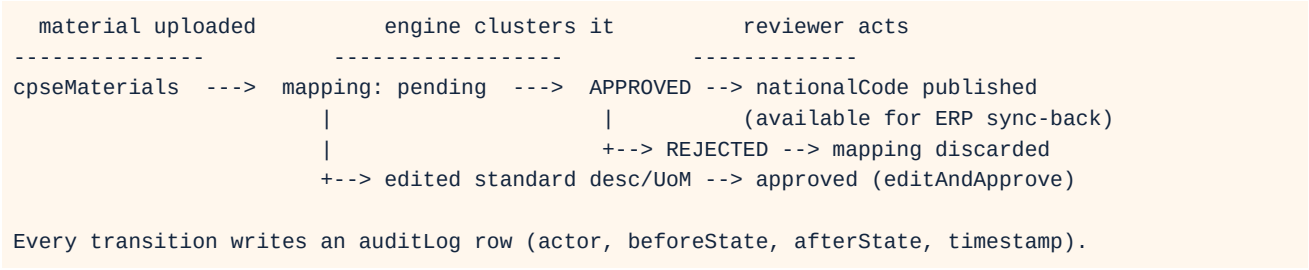
• **Idempotent:** already-mapped materials are skipped, so re-running matching after new imports only processes the delta.

• **Human-in-the-loop:** the engine never publishes anything — it drafts clusters and mappings for review.

# 4. Prototype Modules (what each screen does)

| Module / Route | Technical behavior |
|---|---|
| / · Overview (Landing) | Live stats from Convex (materials ingested, duplicate clusters, national codes, pending reviews) + workflow explainer. Hero sits over the refinery-dusk backdrop. |
| /materials | Full material catalog with free-text search, CPSE filter, mapping-status badges (ingested/mapped/review/approved), and a one-click 'Run AI Matching' action with toast feedback. |
| /review-queue | Pending mappings with confidence badges (green ≥85%, amber 70–84%, red <70%), side-by-side material comparison, cluster siblings, inline edit-and-approve of the standard description/UoM, approve/reject mutations. |
| /analytics | Category breakdown bar chart of duplicate clusters, interactive procurement-savings calculator ((duplicates – 1) × assumed unit cost, formatted in ■ Lakh/Crore), and a recent-activity feed from the audit log. |
| /audit-trail | Searchable, filterable (action/entity/actor) audit log with expandable rows showing full before/after JSON diffs, sortable by timestamp — the compliance backbone. |
| /ingestion | Drag-and-drop CSV upload (PapaParse), column validation with row-level warnings, preview table, bulk insert, then optional one-click AI matching on the fresh rows. |
| /integration | Per-CPSE ERP cards (SAP RFC/BAPI, REST, OData simulated) with sync progress, per-record sync state (syncedAt), a 'Sync All' orchestrator, and JSON payload previews of what would be pushed to each ERP. |

## 4.1 Approval state machine



# 5. How This Solves the Problem Statement — Impact Mapping

| Problem (SIH 26099) | ScrewIT's answer | Evidence in prototype |
|---|---|---|
| 15–30% duplicate material masters across CPSE catalogs | AI matching engine detects cross-CPSE duplicates (exact, near-duplicate, functional-equivalent) and consolidates them under one national code. | Materials + Review Queue screens; cluster counts on Overview/Analytics. |
| No aggregate-demand visibility → lost bulk-procurement leverage | National code master gives a single view of who holds what; the savings calculator quantifies consolidated procurement gains in ■. | Analytics → 'Estimated Procurement Savings' with editable unit-cost assumption. |
| Manual, error-prone cross-CPSE reconciliation | The engine drafts candidates with confidence scores; experts approve/edit/reject in a guided queue instead of eyeballing spreadsheets. | Review Queue side-by-side comparison with confidence badges. |
| Legacy ERP systems cannot be disrupted | Non-invasive by design: CPSE codes stay untouched; national codes are added as cross-reference mappings and synced back via connectors. | System Integration screen with per-ERP sync simulation + payload preview. |
| Government-grade accountability | Append-only audit log with before/after JSON for every action; nothing is deleted. | Audit Trail screen with filters and JSON diff viewer. |

## 5.1 Measurable outcomes the dashboard tracks

- Total materials ingested across CPSEs
- Duplicate clusters detected and national codes approved
- Pending reviews (workflow health)
- Estimated procurement savings (■ Lakh / ■ Crore) at an assumed unit cost
- Sync status per CPSE ERP (approved vs synced vs pending)

## 5.2 Running the prototype

```
# prerequisites: Node.js >= 20.19 (or >= 22.12), npm
npm install
npm run dev:all      # Convex backend + Vite frontend together
npm run seed         # load demo CPSE data with real duplicate clusters

# app:  http://localhost:5173
# then: Materials -> 'Run AI Matching' -> Review Queue -> approve
#       Analytics shows clusters + savings; Integration simulates ERP sync
```

## 5.3 Production hardening path (prototype → deployment)

- **Scale-out matching:** move the pairwise/cluster stage to a Python (FastAPI) worker with sentence-transformer embeddings + pgvector ANN search; the Convex action becomes the orchestrator.
- **Real ERP connectivity:** replace simulated connectors with SAP RFC/BAPI (BAPI_MATERIAL_SAVEDATA custom field) and Kafka topics (material.created / material.approved) for near-real-time delta sync.
- **Auth & RBAC:** Keycloak (OIDC) integrated with CPSE AD/SSO; role-scoped queues per CPSE; maker-checker on approvals.

• **Security & residency:** data-at-rest encryption, Indian data-centre deployment (Meity-empanelled cloud), DPDP-compliant access logging.

---